# On Solving the Multiple Variable Gapped Longest Common Subsequence Problem

Marko Djukanović[1,2,6], Nikola Balaban[2], Christian Blum[3], Aleksandar Kartelj[4], Sašo Džeroski[5], and Žiga Zebec[6]

[1] University of Nova Gorica, Nova Gorica, Slovenia
marko.dukanovic@ung.si
[2] Faculty of Natural Sciences and Mathematics, University of Banja Luka, Banja Luka, Bosnia and Herzegovina
nikola.balaban@student.pmf.unibl.org, marko.djukanovic@pmf.unibl.org
[3] Artificial Intelligence Research Institute (IIIA-CSIC), Barcelona, Spain
christian.blum@iia.csic.es
[4] Faculty of Mathematics, University of Belgrade, Belgrade, Serbia
kartelj@matf.rs
[5] Jožef Stefan Institute, Ljubljana, Slovenia
saso.dzeroski@ijs.si
[6] Institute of Information Sciences (IZUM), Maribor, Slovenia
ziga.zebec@izum.si

**Abstract.** This paper addresses the Variable Gapped Longest Common Subsequence (VGLCS) problem, a generalization of the classical LCS problem involving flexible gap constraints between consecutive solutions' characters. The problem arises in molecular sequence comparison, where structural distance constraints between residues must be respected, and in time-series analysis where events are required to occur within specified temporal delays. We propose a search framework based on the root-based state graph representation, in which the state space comprises a generally large number of rooted state subgraphs. To cope with the resulting combinatorial explosion, an iterative beam search strategy is employed, dynamically maintaining a global pool of promising candidate root nodes, enabling effective control of diversification across iterations. To exploit the search for high-quality solutions, several known heuristics from the LCS literature are utilized into the standalone beam search procedure. To the best of our knowledge, this is the first comprehensive computational study on the VGLCS problem comprising 320 synthetic instances with up to 10 input sequences and up to 500 characters. Experimental results show robustness of the designed approach over the baseline beam search in comparable runtimes.

**Keywords:** Common subsequences · Beam Search · Bioinformatics · Gap constraints

## 1 Introduction

The *Longest Common Subsequence Problem* (LCSP) [1, 4] is a well-known combinatorial optimization problem with numerous applications in bioinformatics and

computational biology, playing a fundamental role in the analysis and comparison of molecular sequences. Given an arbitrarily large set of input sequences $S = \{s_1, \ldots, s_m\}, m \in \mathbb{N}$ over an alphabet $\Sigma$, the aim is to identify the longest possible subsequence that is common to all sequences $s_i \in S$. Over the past decades, a variety of practically motivated extensions of the LCSP have been proposed to better capture structural, biological, or application-specific requirements of real-world sequences. Notable variants include constrained, arc-preserving, and repetition-free longest common subsequence problems [6, 5, 3], among others. In this work, we focus on yet another practical variant, the *Variable Gapped LCS Problem* (VGLCSP), originally introduced for the $m = 2$ case in [9], later investigated from a theoretical perspective in [2]. The VGLCSP extends the classical LCSP by incorporating flexible distance constraints (referred to as *gaps*) between consecutive symbols of the resulting subsequences and their respective positions in the input sequences. Unlike fixed-gap models, these gap limits are allowed to vary along the input sequences, thereby offering increased modeling flexibility.
Formally, for each input sequence $s_i \in S$, its gap constraint is defined as a function over the positions of the sequence, i.e., $G_i \colon \{1, \ldots, |s_i|\} \mapsto \mathbb{N}$. Further, if the characters of a solution $s$ occur at positions $i_i^1 < \cdots < i_i^{|s|}$ in $s_i$, the gap constraint is satisfied iff $i_i^x - i_i^{x-1} \leq G_i[i_i^x] + 1, \quad x = 2, \ldots, |s|$. A common subsequence $s$ is considered *feasible* if the corresponding gap constraints are satisfied simultaneously for all sequences in $S$. As an example of a VGLCSP instance, given is a set $S = \{s_1 = \texttt{ABCA}, s_2 = \texttt{ACAB}\}$ with two gap constraints $G_1(\{1, \ldots, 4\}) \mapsto \mathbf{1} = G_2(\{1, \ldots, 4\})$. The sequence $s = \texttt{ACA}$ is a feasible solution because it is a common subsequence for both input strings, where its corresponding characters appear at positions 1, 3, and 4 in $s_1$ and at the positions 1, 2, and 3 in $s_2$. As the difference between each consecutive pair of the indices of these positions is less than or equal to $1+1 = 2$, $s$ represents a feasible sequence of the tackled problem. On the other hand, sequence $s = \texttt{AB}$ represents a common subsequence, but not fulfilling all gap constraints because the corresponding characters appear at positions 1 and 2 in $s_1$ and at positions 1 and 4 in $s_2$. Obviously, $4-1 = 3 > 2 = 1+1$, thus violating the second gap constraint.
The VGLCSP provides a flexible and realistic model for sequence comparison, particularly suiting to applications in DNA and protein analysis where variable structural distances between (active) residues must be respected. Beyond bioinformatics, the problem also arises in time-series analysis, especially in settings where events are required to occur within specified temporal delays [7].
Although several dynamic programming approaches exist for the $m = 2$ case [9], they are computationally prohibitive when expanded for larger $m$. To the best of our knowledge, no effective exact or approximate approaches are designed for the generalized VGLCSP with arbitrary $m$, clearly an $\mathcal{NP}$-hard problem as the generalization of LCS problem with an arbitrarily large set of input strings.

*Preliminaries.* Let $|s|$ denote the length of a sequence $s$, and let $s[i]$ refers to the character of $s$ at position $i$, where indexing starts at $i = 1$. The substring of $s$ that begins at position $i$ and ends at position $j$ is denoted by $s[i, j]$. If $i = j$, this corresponds to the single character $s[i]$; if $j < i$, then $s[i, j] = \varepsilon$,

where $\varepsilon$ denotes the empty string. The number of occurrences of $a \in \Sigma$ in $s$ is denoted by $|s|_a$. We denote by $S = \{s_1, \ldots, s_m\}$ the set of input sequences, and by $S^{rev} = \{s_1^{rev}, \ldots, s_m^{rev}\}$, the set of reverse strings. Let $m \in \mathbb{N}$ be the number of input sequences (and, correspondingly, the number of gap constraints), and $n$ denotes the length of the longest sequence in $S$. Given a positional vector $\mathbf{p}^L \in \mathbb{N}^m$, the subproblem related to these positions is given by $S[\mathbf{p}^L] = \{s_i[\mathbf{p}_i^L, |s_i|] \mid i = 1, \ldots, m\}$. Last but not least, for a gap constraint $G_i$, its reverse constraint is denoted by $G_i^{rev}$, given by $G_i^{rev}(j) := G_i(|s_i| - j + 1)$, for each $j = 1, \ldots, |s_i|$.

The remainder of the paper is organized as follows. Section 2 introduces the rooted graph representation of the problem. In Section 3, we design our main methodological contribution, the iterative multi-source beam search approach. Section 4 reports the computational results, comparing the proposed method with a baseline beam search and an iterative greedy heuristic. Finally, Section 5 concludes the paper and outlines directions for future research.

## 2 Rooted State Space Formulation

Motivated by the state-space representation for the LCSP introduced in [1], we design a *rooted state graph* model for the VGLCSP. Each state represents one or more feasible partial solutions characterized by a vector of positions induced by the input sequences and by the current subsequence length.

Formally, a partial subsequence $s^v$ induces a state node $v = (\mathbf{p}^{L,v}, l^v)$, where $\mathbf{p}^{L,v} = (p_1^{L,v}, \ldots, p_m^{L,v})$ and the following conditions hold: (i) for each sequence $s_i \in S$, $p_i^{L,v}-1$ is the smallest position such that $s^v$ is a subsequence of $s_i[1, p_i^{L,v}-1]$; (ii) $l^v = |s^v|$ denotes the length of the partial subsequence; and (iii) all gap constraints $G_i$ are *satisfied* for $s^v$ with respect to each input sequence $s_i$. A direct arc $\alpha = (v_1, v_2)$, labeled with $\texttt{lett}(\alpha) = a \in \Sigma$, exists if $l^{v_2} = l^{v_1} + 1$ and $s^{v_2} = s^{v_1} \cdot a$; that is, state $v_2$ corresponds to extending a partial solution of $v_1$ by (appending) character $a$, respecting all gap constraints.

To expand a state $v$, those characters $a \in \Sigma$ that occur in each suffix $s_i[p_i^{L,v}, |s_i|]$, $i = 1, \ldots, m$, are first detected. For each such $a \in \Sigma$, we identify the smallest feasible positions $p_i^{L,a} \geq p_i^{L,v}$ such that $s_i[p_i^{L,a}] = a$ and $p_i^{L,a} - p_i^{L,v} \leq G_i(p_i^{L,a})$, $i = 1, \ldots, m$. In that way, a child state $w = (\mathbf{p}^{L,a} + \mathbf{1}, l^v + 1)$ is created, unless it is dominated by another child state according to the dominance criteria defined later in this section. To efficiently compute children and their associated position vectors, we employ a preprocessing data structure denoted by `Succ`: a three-dimensional integer array indexed by $(i, j, a)$, where $i$ refers to the input sequence $s_i$, $1 \leq j \leq |s_i|$ denotes a position within that sequence, and $a \in \Sigma$. The value $q = \texttt{Succ}[i, j, a]$ represents the smallest index $q \geq j$ such that $s_i[q] = a$ so that the gap constraint $G_i$ is satisfied at position $q$, i.e., $q - j \leq G_i(q)$. If no such position exists, $q = -1$ is assigned. Goal states are those that cannot be further expanded. The trivial root state is given by $r = ((1, \ldots, 1), 0)$ as in-going edges to this node are not possible, representing an empty partial solution like

in the case of the LCS problem. However, this is not the only one root node, in general. The state graph of a problem can have many (possibly exponentially) root nodes, detecting the essential decision in selecting leading letter for future expansions. Note that if match $u = (i_1^1, \ldots, i_m^1)$ represents a starting position, the gap constraints do not apply to the corresponding letter of this or any other rooted match.

This motivate us to introduce the state space subgraph rooted with match $\mathbf{p}^{L,u}$, denoted by $\textsc{RootedSubgraph}(u = (\mathbf{p}^{L,u}, 0))$, where $u$ is induced by empty partial solution. The empty solution is initially expanded by the letter $l$ matching $\mathbf{p}^{L,u}$ as we require to generate all feasible solutions having $l$ for the leading letter. In essence, this subgraph consists of all nodes (and corresponding transitions) feasibly reached via node $(p^{L,u} + \mathbf{1}, 1)$. These nodes are associated to partial solutions with a leading letter to match position $\mathbf{p}^{L,u}$. We recursively repeat the procedure of expanding solutions by one letter in all possible ways, until no feasible expansions are possible. As discussed in the following section, the structure of this subspace heavily depends on the choice for root state $u$ from which the search is initiated. We illustrate the space of $\textsc{RootedSubgraph}(u = ((1,1),0))$, depicted in Fig. 1. Note that here the position $(1,1)$ correspond to the match of letter `A` associated to $u = ((1,1),0)$ which is indeed a root node.

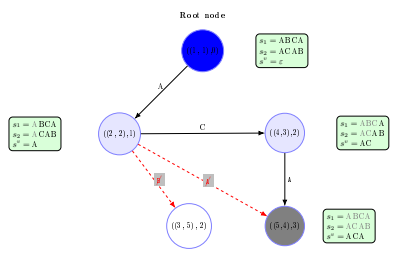



Fig. 1: $\textsc{RootedSubspace}(u = ((1,1),0))$ between $\boxed{\texttt{A}}$`BCA` and $\boxed{\texttt{A}}$`CAB`, assuming $G_1 = G_2 = \mathbf{1}$. A final/terminal state is displayed in gray. The white-filled node is a valid LCS node, but not an VGLCS node—the second gap constraint is violated. The longest solution generated from this subspace is `ACA`.

*Multi-Source Beam Search.* Unlike the classical LCSP, the VGLCSP may exhibit multiple, potentially exponentially many, *disconnected root components* in the full state space. Consequently, a search strategy that expands states from a single initial root node may fail to reach optimal solutions.

As an illustrative example given in Fig. 2, consider the sequence set $S = \{s_1 = \texttt{ATGGAAA},\ s_2 = \texttt{ATCCAAA}\}$, with $G_{s_1} = G_{s_2} = 1$. In this instance, any state with position vector $\mathbf{p}^L = (5,5)$ is not reachable from the initial state $((1,1),0)$ by a series of direct transitions. As a result, the optimal common subsequence `AAA` is unreachable from $((1,1),0)$ when exploring its subgraph. To address this issue, we define the complete state space of a VGLCSP instance by $\bigcup_{v \in \mathcal{R}} \textsc{RootedSubspace}((\mathbf{q}^{L,v}, 0))$, where $\mathcal{R}$ denotes the set of all root states, which cannot be reached from any other state by a series of direct transitions. However, explicitly enumerating all such root states is computationally prohibitive and in generally requires $O(n^m)$ time, thus remains infeasible

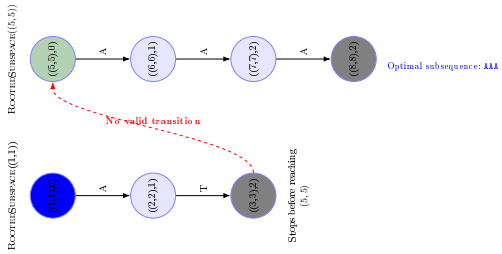



Fig. 2: Root state space subgraphs: disconnected components.

for instances with large $m$ or $n$. This structural characteristic of the problem itself motivates the methodology entitled *Iterative Multi-Source Beam Search* (IMSBS) approach, specifically designed to dynamically explore multiple promising rooted subregions of the state space by systematically moving exploration from one subgraph to another one. By iteratively identifying and expanding a set of candidate root states, IMSBS effectively mitigates the disconnection effects induced by gap constraints, enabling the discovery of high-quality solutions that would otherwise remain inaccessible to single-source search methods.

## 3 Iterative Multi-source Beam Search Framework

*Beam search* (BS) is a well-established tree-search-based metaheuristic that performs in a breadth-first-search manner. Its search is controlled by a beam width parameter $\beta$—specifying the maximum number of nodes retained for expansion at each level—and by heuristic guidance $h$— which guides the selection of the most promising candidates for further expansion. Initially, the procedure receives a set of nodes as the initial beam. After being evaluated, up to $\beta$ most promising nodes are kept for creating a beam of the subsequent level. The procedure repeats until the beam is empty, returning the longest partial solution.

In the IMSBS, the beam search component serves to intensify the search process. As the search space of a VGLCSP instance is represented by the set of root nodes $\mathcal{R}$, the BS procedure is in charge of efficiently exploring various regions $\textsc{RootedSubspace}((\mathbf{p}^L, 0))$ induced by corresponding root nodes. Note that one could here employ an exact technique (such as A$^*$ or BFS); however, solving the subproblem itself could be a resource-demanding task, as hard as the original problem itself. Thus, we opted to utilize a more robust BS technique.

The main core of the IMSBS framework is summarized as follows. Initially, the global pool of candidate root nodes is initialized to an empty set $\mathcal{R} := \{\}$ and the current best solution to $s_{\text{best}} := \varepsilon$. First, for each $a \in \Sigma$, the closest matches $\mathbf{p}^{L,a}$ from the leading positions of each input sequence are detected. Each such (non-dominated) match generates one root node $(\mathbf{p}^{L,a}, 0)$, which has been subsequently added to $\mathcal{R}$. The core iterations of IMSBS have been executed until either $\mathcal{R}$ becomes empty, a predefined maximum number of iterations has reached, or allowed time limit has exceeded. At each iteration, a fixed number (*sources_from_R*) of most promising candidate root nodes according to a

heuristic $h'$ is taken from $\mathcal{R}$, and subsequently stored in a temporary beam $\mathcal{L}$. These nodes are refined, executing a single backward beam search procedure for each $w \in \mathcal{L}$ as its starting node $(\mathbf{p}^w, 0)$. The procedure works similarly to the baseline BS, expanding nodes according to the gap constraints but in the reversed manner, starting from the root positions at each sequence and moving the search (pointers) towards their leading positions. In essence, for each $w \in \mathcal{L}$, a beam search is executed on the reversed input sequences $S^{rev}$ and the reversed gap constraints $(G_i^{rev})_{i=1}^m$ with the initial beam width $B = \{((|s_i| - p^w)_{i=1}^m, 0)\}$, beam size $\beta'$ and heuristic guidance $h'$. Assume the procedure has completed, returning a solution $s_{bwrd}^w$. In this way, the reverse sequence $s_{bwrd}^{rev}$ represents a feasible VGLCS solution and the partial solution without the last letter—which corresponds to the match $\mathbf{p}^w$—is assigned to the refined node $w \in \mathcal{L}$, and the updated node $w = (p^w, |(s_{bwrd}^w)^{rev}| - 1)$ is passed to the forward beam search procedure for the forward-pass beam search. Thus, the refined set $\mathcal{L}$ enters in as the initial beam, executing a BS parametrized with $\beta$ and $h$, following the subspace definition in Section 2. During this phase, all complete (non-expandable) nodes are stored in the list $V_{complete}$, which is, after termination, returned along with the best found solution $s_b$. Each complete node from $V_{complete}$ is then independently expanded—ignoring the gap constraints— to generate a set of candidate root nodes. In details, for each complete node $v = (\mathbf{p}^v, l^v)$, candidate nodes $r^{w,a} = (\mathbf{p}^{w,a}, 0)$ are generated for each letter $a \in \Sigma$, where $p_i^{w,a}$ represents the match position of that letter in $s_i$ which is closest from position $\mathbf{p}_i^v$. Each such candidate node $(r^{w,a}, 0)$ is added to $\mathcal{R}$, given that it has not already been part of $\mathcal{R}$ during the previous iterations. Once the BS procedure is terminated, returned solution if verified as a new incumbent, and the next IMSBS iteration is initiated with a newly selected set of most promising root nodes from $\mathcal{R}$, until a termination condition is met. The pool $\mathcal{R}$ may be maintained as a priority queue implemented via a binary heap, to ensure efficient extraction of the most promising candidate root nodes.

The core advantages of the IMSBS approach over the baseline BS are: (i) A provision of balance between complete beam search execution (local exploration of promising search subregions) and the generation of new candidate root nodes (global coverage of the search space); (ii) Prevention from suboptimal candidate root nodes is achieved by executing the backward beam search procedure.

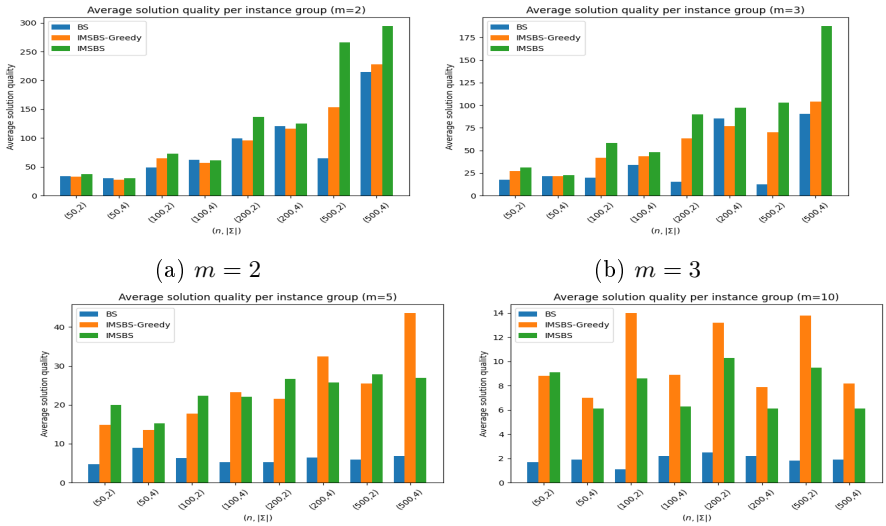
*Heuristic guidances.* Several heuristic guidances to guide search are utilized, known from the LCS literature: (i) *"Look-ahead" for the remaining length:* $\mathrm{UB}_1(\mathbf{v}) = l^v + \min_{i=1,\ldots,m}\left(|s_i| - p_i^{L,v} + 1\right)$; (ii) *Character Frequency Alignment:* $\mathrm{UB}_2(\mathbf{v}) = l^v + \sum_{\sigma\in\Sigma} \min\left(|s_1[p_1^{L,v}, |s_1|]|_\sigma, \ldots, |s_m[p_m^{L,v}, |s_m|]|_\sigma\right)$, counting the maximum number of characters that can still match in the (optimal) solution based on the frequency of each character; (iii) *A probability-based heuristic* guidance $h_{prob}$ by the Mousavi and Tabataba's matrices [8]: These probabilities are based on determining a probability of the event realization that a sequence $s$ of length $k$ is a subsequence of a (random) sequence of length $n$ over $\Sigma$. More about these heuristics interested readers can found at [1, 8].

## 4 Experimental Evaluation

The performance of heuristic approaches designed for an arbitrary $m \geq 2$ are experimentally compared. Specifically, the following methods are included: (i) Bs: a baseline beam search, which performs a single iteration of IMSBS while employing a huge beam width; (ii) Imsbs-greedy: it sets a fixed $\beta = 1$ employing a huge number of IMSBS iterations; (iii) Imsbs: a tuned IMSBS with an average runtime comparable to that of Bs. The Imsbs is implemented in Python 3.11 and evaluated on the VEGA HPC system hosted at IZUM, Maribor. The cluster consists of 960 compute nodes equipped with AMD EPYC 7H12 CPUs operating at 2.35 GHz. All experiments were conducted in a single-threaded setting. For each combination of $n \in \{50, 100, 200, 500\}$, $m \in \{2, 3, 5, 10\}$, and $|\Sigma| \in \{2, 4\}$, 10 random problem instances are generated. First, $m$ sequences of equal length are generated uniformly at random. Then, the gap constraints for each instance are generated such that each value $G_i(j), j = 1, \ldots, |s_i|, i = 1, \ldots, m$ is assigned uniformly from $\mathcal{U}(\{\lfloor 0.5 \cdot |\Sigma| \rfloor, \ldots, \lfloor 1.5 \cdot |\Sigma| \rfloor\})$. Therefore, a total of *320 problem instances* is generated. The benchmark set is denoted as Random, freely available at `https://github.com/markodjukanovic90/NikolaBalabanDiplomski/tree/main/src/instance_i_generator`.

*Parameter tuning.* To tune the parameters of our algorithms, we employed a basic grid search to more sensible parameters: beam width in the beam search forward pass ($\beta$), and heuristic guidance $h \in \{\mathrm{UB}_1, \mathrm{UB}_2, h_{prob}\}$, comparing the average solution quality over all (320) problem instances. Less sensible parameters are selected according to several preliminary runs. We fix $sources_from_R = 10$ and $beam_iters = 100$ (beam iterations in Imsbs), and $h' = \mathrm{UB}_2$. For the baseline Bs approach, a high beam width $\beta = 10,000$ and $h = h_{\mathrm{prob}}$ are utilized. Remaining parameters of the Imsbs approach received the following values: $h = \mathrm{UB}_2$ and $\beta = 500$ to keep comparable average runtime between Imsbs and the baseline Bs approach. These values balance runtime relative to Bs while maintaining a reasonable solution quality. Similarly, for the Imsbs-greedy approach with fixed $\beta = 1$ uses $beam_iters = 10,000$, and all other parameters hold as in the case of tuned Imsbs.

Numerical results for all three heuristic approaches are displayed in Fig. 3, reporting the average solution quality per instance group for each approach. Four plots are provided, one per each value of $m$. The Imsbs approach produces the highest average solution quality in 21 out of 32 instance groups, surpassing the baseline Bs approach dramatically, indicating that the selection of the right root node is essential in achieving high-quality solutions. Bs performs better in just one case. Concerning the Imsbs-greedy, it surpasses the other two approaches in 10 (out of 32) cases, mostly for the instances with larger $m = 10$, having shorter final solutions where it turns out that frequently moving from one to another root and exploring various subregions of the search space is strategically crucial than just invest the time for strengthening the search around a solution employing a hige $\beta$. Exploring additional root nodes improves the diversity of the search, which appears necessary in cases where expected solutions are short. This is exactly where Imsbs-greedy fits well. Bs and Imsbs approaches are tuned




(a) $m = 2$ (b) $m = 3$

(c) $m = 5$ (d) $m = 10$

Fig. 3: Average solution quality of the three approaches.

to exhibit comparable average runtimes. IMSBS-GREEDY has a larger average runtimes for the instances with larger $m$ due to a higher number of executed iterations ($beam_iters = 10,000$ iterations allowed) which are not cheap when $m$ is large. Detailed numerical results with the accompanying plots can be found in the supplementary material at `https://github.com/markodjukanovic90/VGLCS/blob/main/doc/EUROCAST-paper/supplementary.pdf`.

## 5 Conclusions and Future Work

This paper studied a generalized variable gapped LCS problem. It extends the classical LCS problem by constraining the allowable distances between matched characters, reflecting biological scenarios in which spatially closer nucleotides interact more strongly than distant ones. We proposed an Iterative Multi-Source Beam Search (IMSBS), which performs local exploration from promising root nodes involving a global strategy of candidate root-node selection based on previously collected search information. Experimental results indicate the IMSBS consistently produces the highest-quality solutions over the baseline beam search. In cases where shorter expected solutions are anticipated, frequently transitioning between different feasible subspaces is more effective. This is in contrast to situations with a small sequence number, where a larger beam width and a greater number of IMSBS iterations are necessary to obtain high-quality solutions. Future research includes designing more sophisticated heuristics which incorporate gap constraints into scoring, optimizing IMSBS to efficiently reuse previously explored regions, and testing the approaches on real-world biological instances.

***Acknowledgments.*** This publication is co-funded by the European Union's Horizon Europe research and innovation program under the Marie Sklodowska-

Curie COFUND Postdoctoral Programme (grant agreement No. 101081355 – SMASH), and by the Republic of Slovenia and the European Union through the European Regional Development Fund. Views and opinions expressed are those of the authors only and do not necessarily reflect those of the European Union or the European Research Executive Agency (REA). Neither the European Union nor the REA can be held responsible for them.

## References

1. Finding longest common subsequences: New anytime A* search results. Applied Soft Computing **95**, 106499 (2020)
2. Adamson, D., Kosche, M., Koß, T., Manea, F., Siemer, S.: Longest common subsequence with gap constraints. In: International Conference on Combinatorics on Words. pp. 60–76. Springer (2023)
3. Adi, S.S., Braga, M.D., Fernandes, C.G., Ferreira, C.E., Martinez, F.V., Sagot, M.F., Stefanes, M.A., Tjandraatmadja, C., Wakabayashi, Y.: Repetition-free longest common subsequence. Discrete Applied Mathematics **158**(12), 1315–1324 (2010)
4. Bergroth, L., Hakonen, H., Raita, T.: A survey of longest common subsequence algorithms. In: Proceedings Seventh International Symposium on String Processing and Information Retrieval. SPIRE 2000. pp. 39–48. IEEE (2000)
5. Farhana, E., Rahman, M.S.: Constrained sequence analysis algorithms in computational biology. Information Sciences **295**, 247–257 (2015)
6. Jiang, T., Lin, G., Ma, B., Zhang, K.: The longest common subsequence problem for arc-annotated sequences. Journal of Discrete Algorithms **2**(2), 257–270 (2004)
7. Lainscsek, C., Sejnowski, T.J.: Delay differential analysis of time series. Neural computation **27**(3), 594–614 (2015)
8. Mousavi, S.R., Tabataba, F.: An improved algorithm for the longest common subsequence problem. Computers & Operations Research **39**(3), 512–520 (2012)
9. Penga, Y.H., Yangb, C.B.: The longest common subsequence problem with variable gapped constraints. In: Proceedings of the 28th Workshop on Combinatorial Mathematics and Computation Theory, Penghu, Taiwan. pp. 17–23 (2011)